

Experiment 1

Aim:

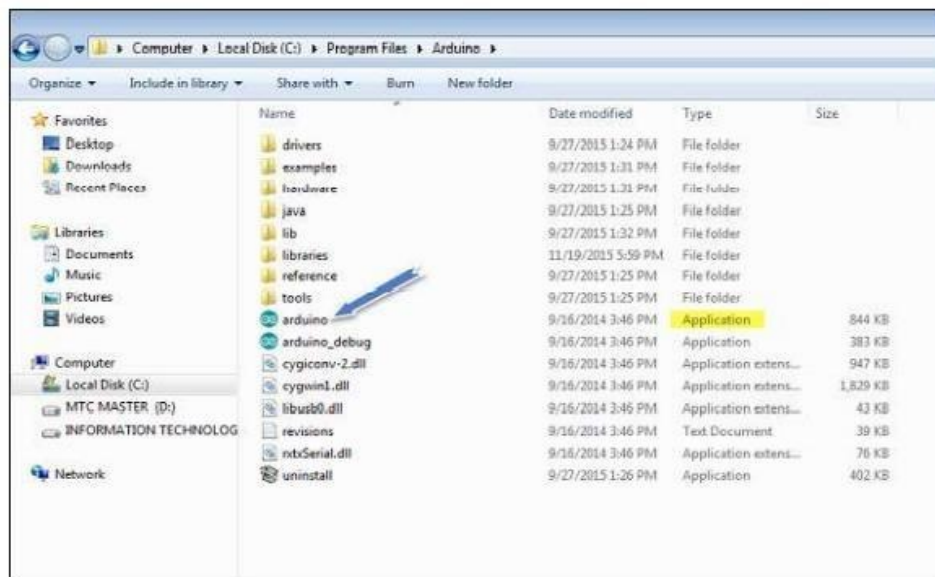
To familiarize myself with Arduino and perform the necessary software installation.

Description:

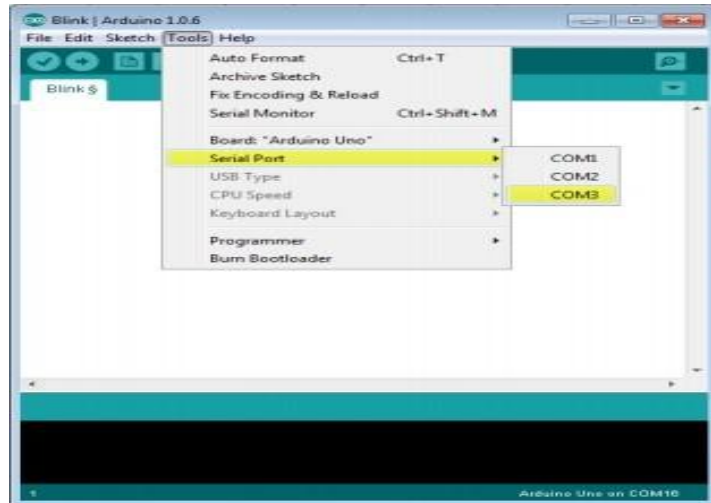
In this experiment, I gained knowledge about the Arduino and Raspberry Pi platforms. I installed the Arduino IDE software, familiarized myself with its interface, and successfully ran a sample program. This experiment helped me learn how to configure the development environment and upload a simple program to the Arduino board.

Procedure:

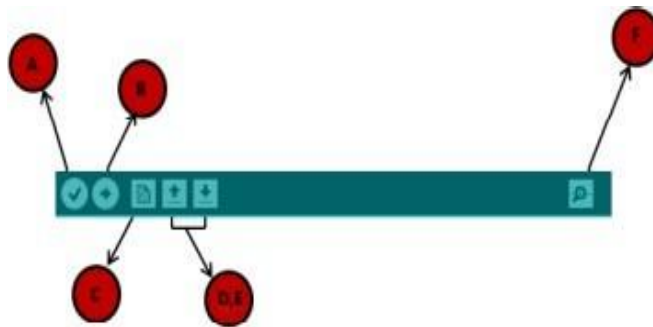
1. I downloaded the Arduino IDE software from the official website <https://www.arduino.cc>.
2. I launched the Arduino IDE on my system.
3. After the download, I unzipped the folder and located the application file with the infinity label (application.exe). I double-clicked on the icon to start the IDE.



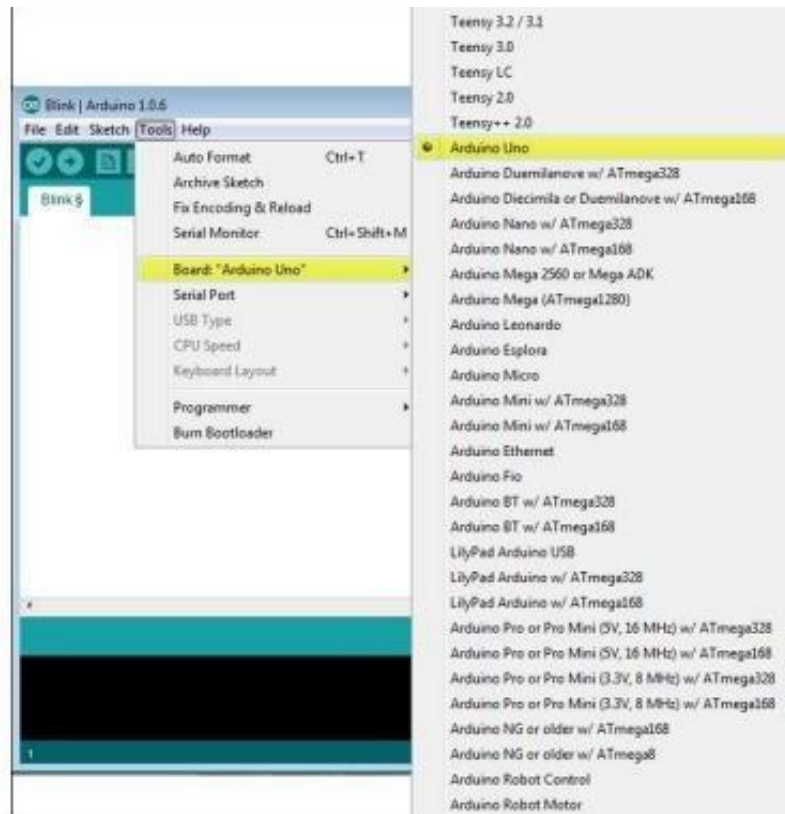
4. Once the IDE opened, I had two options: create a new project or open an existing example. I chose to open an existing project by navigating to **File** → **Examples** → **Basics** → **Blink**. This program makes the LED turn on and off with a time delay.



5. I selected my Arduino board by going to **Tools** → **Board** and choosing **Arduino Uno**, as it matched the board I was using.
6. I selected the correct serial port by navigating to **Tools** → **Serial Port**. I verified this by disconnecting and reconnecting my Arduino board and then choosing the correct COM port.
7. I became familiar with the Arduino IDE toolbar functions:



- a. Verify (check for errors)
 - b. Upload (send program to the board)
 - c. New Sketch (create a new project)
 - d. Open (open existing examples)
 - e. Save (save the sketch)
 - f. Serial Monitor (communicate with the board)
8. clicked the **Upload** button to send the Blink program to the Arduino board.



9. observed the RX and TX LEDs flashing on the board. After a few seconds, the message **“Done uploading”** appeared in the status bar.

Result:

I successfully installed the Arduino IDE, configured the board and serial port, and uploaded the Blink program. The Arduino Uno executed the program correctly, and the onboard LED blinked with a time delay.

Conclusion:

Through this experiment, I understood how to install and set up the Arduino IDE, select the appropriate board and port, and upload a program to the Arduino. This gave me the basic knowledge required to start working on IoT applications.

Experiment 2

Aim:

To interface an LED/Buzzer with Arduino and write a program to turn ON the LED for 1 second after every 2 seconds.

Requirements:

- ☐ Standard 5mm LED
- ☐ One Resistor
- ☐ Arduino Uno microcontroller board (ATmega328p)
- ☐ Jumper wires

Description:

In this experiment, I connected an LED to the Arduino Uno microcontroller using a resistor for protection. I programmed the Arduino to control the LED through a digital pin. The LED remained ON for 1 second and OFF for 2 seconds continuously. The program was executed in a repeated loop to make the LED blink. This experiment helped me understand how digital pins control external devices in IoT applications.

Procedure:

1. I opened www.wokwi.com in my browser and selected the **Arduino Uno** microcontroller.
2. In the simulation section, I added the required components (LED, resistor, jumper wires, and Arduino Uno) by clicking the “+” symbol.
3. I used jumper wires to make the connections: I connected the **Anode (A-pin)** of the LED to **digital pin 3** of the Arduino board through the resistor, and I connected the **Cathode (C-pin)** of the LED to **Ground (GND)**.
4. I wrote the following program in the **sketch.ino** file:

Code:

```
/*  
  Experiment: Interfacing an LED with Arduino Uno  
  Purpose: To turn ON the LED for 1 second after every 2 seconds  
  Name: KIRAN KUMAR PIKKI  
  Roll No: 25M11MC234  
*/  
  
void setup() {  
  pinMode(3, OUTPUT); // Set pin 3 as OUTPUT for LED }
```

```

void loop() {
    digitalWrite(3, HIGH);    // Turn LED ON
    delay(1000);              // Keep ON for 1 second
    digitalWrite(3, LOW);    // Turn LED OFF
    delay(2000);              // Keep OFF for 2 seconds
}

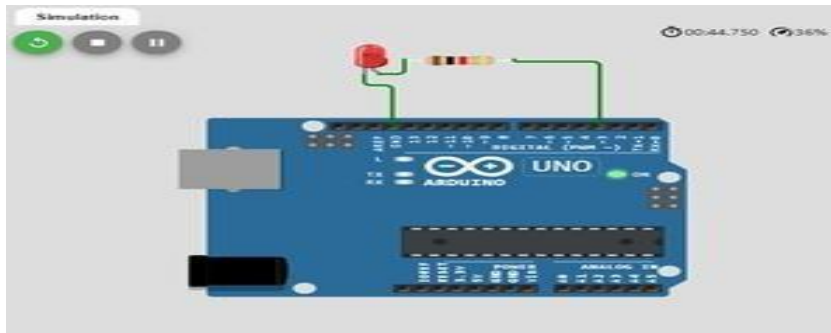
```

5. I compiled and uploaded the program to the Arduino Uno board in the simulation.
6. I observed the LED blinking – it turned ON for 1 second and remained OFF for 2 seconds repeatedly.

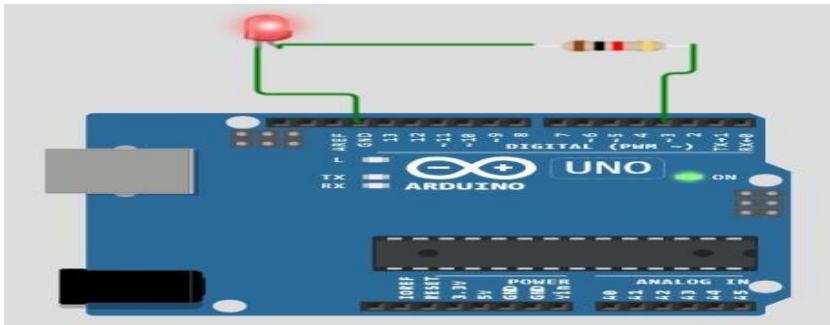
Result:

I successfully interfaced the LED with the Arduino Uno board and observed that the LED turned ON for 1 second and remained OFF for 2 seconds, working as expected.

Before:



After:



Conclusion:

Through this experiment, I learned how to interface external output devices like LEDs with Arduino and control them using digital pins. I also understood how to use `pinMode()`, `digitalWrite()`, and `delay()` functions for timing control in Arduino programs.

Experiment 3

Aim:

To interface a Push Button/Digital Sensor (IR/LDR) with Arduino and write a program to turn ON the LED when the push button is pressed or at sensor detection.

Requirements:

- ☐ Standard 5mm LED
- ☐ Two Resistors
- ☐ Arduino Uno microcontroller board (ATmega328p)
- ☐ Jumper wires
- ☐ Push button

Description:

In this experiment, I connected a push button to the Arduino Uno as a digital input to control an LED. The LED was programmed to turn ON when the button was pressed. This helped me understand how to read digital input signals from a push button. I also learned how input devices can be used to control output devices based on user interaction.

Procedure:

1. I opened www.wokwi.com in my browser and selected the **Arduino Uno** microcontroller.
2. In the simulation section, I added the required components (LED, resistors, jumper wires, push button, and Arduino Uno) by clicking the “+” symbol.
3. I connected the **Anode (A-pin)** of the LED to **digital pin 3** of the Arduino microcontroller through resistor1, and I connected the **Cathode (C-pin)** of the LED to **Ground (GND)**.
4. I provided a **5V power supply** to the push button pin **1:1L**.
5. I connected the push button pin **1:1R** both to **Ground through resistor2** and to **digital pin 5** of the Arduino board.
6. I wrote the following program in **sketch.ino**:

Code:

/*Experiment 3: Interfacing a Push Button/Digital Sensor with Arduino Uno

Purpose: To turn ON the LED when the push button is pressed or at sensor detection

Name: KIRAN KUMAR PIKKI

Roll No: 25M11MC234

*/

```
int flag = 0;
```

```
void setup() {  
  pinMode(3, OUTPUT); // LED connected to pin 3  
  pinMode(5, INPUT);  // Push button connected to pin 5  
}
```

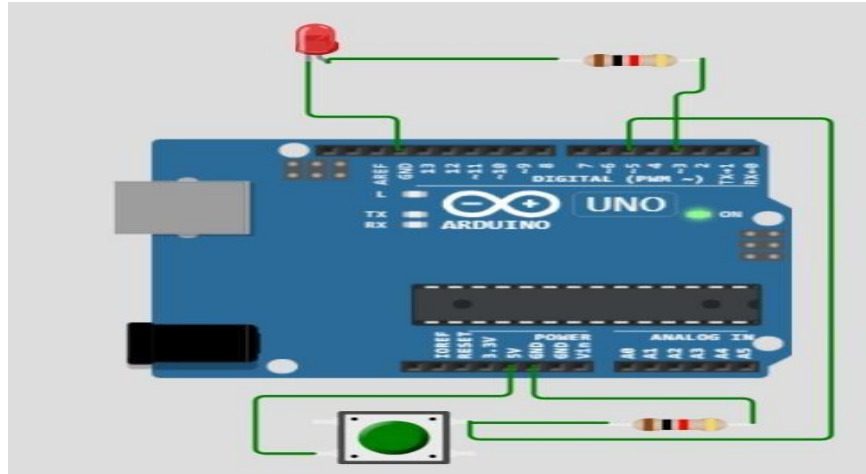
```
void loop() {  
  int value = digitalRead(5); // Read push button state  
  
  if (value == 1 && flag == 0) {  
    digitalWrite(3, HIGH); // Turn LED ON  
    delay(1000);  
    flag = 1;  
  }  
  else if (value == 1 && flag == 1) {  
    digitalWrite(3, LOW); // Turn LED OFF  
    delay(1000);  
    flag = 0;  
  }  
}
```

7. I compiled and uploaded the program to the Arduino Uno board in the simulation.
8. I observed that when I pressed the push button, the LED turned ON. On pressing it again, the LED turned OFF.

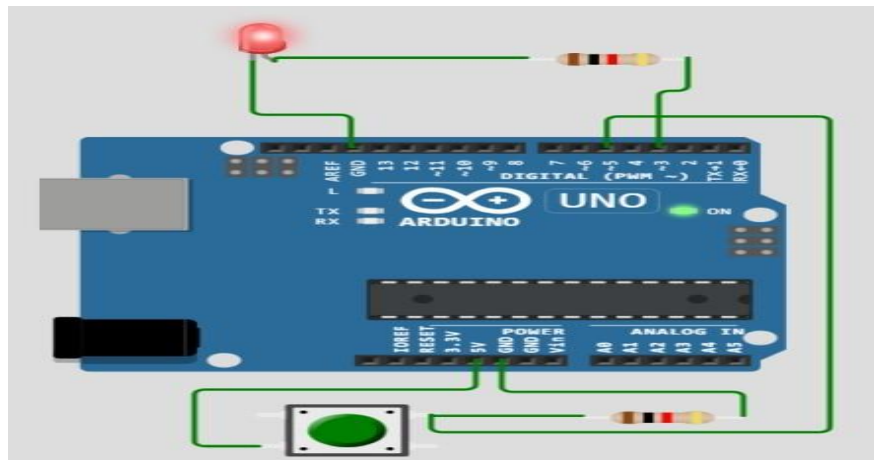
Result:

I successfully interfaced a push button with the Arduino Uno. When I pressed the push button, the LED turned ON, and on pressing again, it turned OFF.

Before:



After:



Conclusion:

Through this experiment, I learned how to interface a push button (digital input) with Arduino and use its state to control an output device. I understood the use of `digitalRead()` for inputs and how conditional logic can toggle device states in IoT applications.

Experiment 4

Aim:

To interface the DHT22 sensor with Arduino and write a program to print temperature and humidity readings.

Requirements:

- ☐ DHT22 sensor
- ☐ Arduino Uno microcontroller board (ATmega328p)
- ☐ Jumper wires

Description:

In this experiment, I connected the DHT22 sensor to the Arduino Uno to monitor temperature and humidity. The sensor provided environmental data and is commonly used in IoT-based applications. This experiment helped me understand the use of sensors for weather monitoring and smart home automation.

Procedure:

1. I opened www.wokwi.com in my browser and selected the **Arduino Uno** microcontroller.
2. In the simulation section, I added the required components (DHT22 sensor, jumper wires, and Arduino Uno) by clicking the “+” symbol.
3. I made the following connections using jumper wires:
 - ☐ I connected **VCC pin of DHT22** to **5V power supply** of Arduino.
 - ☐ I connected **SDA pin of DHT22** to **digital pin 3** of Arduino Uno.
 - ☐ I connected **GND pin of DHT22** to **Ground (GND)** of Arduino Uno.
4. I wrote the following program in **sketch.ino**:

Code:

```
/*  
    Experiment 4: Interfacing the DHT22 sensor with Arduino Uno  
    Purpose: To print temperature and humidity readings  
    Name: KIRAN KUMAR PIKKI  
    Roll No: 25M11MC234  
*/
```

```

#include <Adafruit_Sensor.h>
#include <DHT.h>
#include <DHT_U.h>
DHT Sensor(3, DHT22);

void setup() {
  Serial.begin(9600);
  Sensor.begin();

  // Display student details before starting readings

  Serial.println("=====");
  Serial.println(" Experiment 4: DHT22 Sensor Interface ");
  Serial.println(" Name::KIRAN KUMAR PIKKI");
  Serial.println(" Roll No : 25M11MC234");

  Serial.println("=====");
  delay(2000); // Pause for 2 seconds before readings
}

void loop() {
  float tempr = Sensor.readTemperature(); float
  humid = Sensor.readHumidity();

  Serial.print("Temperature = ");
  Serial.print(tempr);
  Serial.print(" °C, ");
  Serial.print("Humidity = ");
  Serial.print(humid);
  Serial.println(" %");

  delay(1000); // Update every second
}

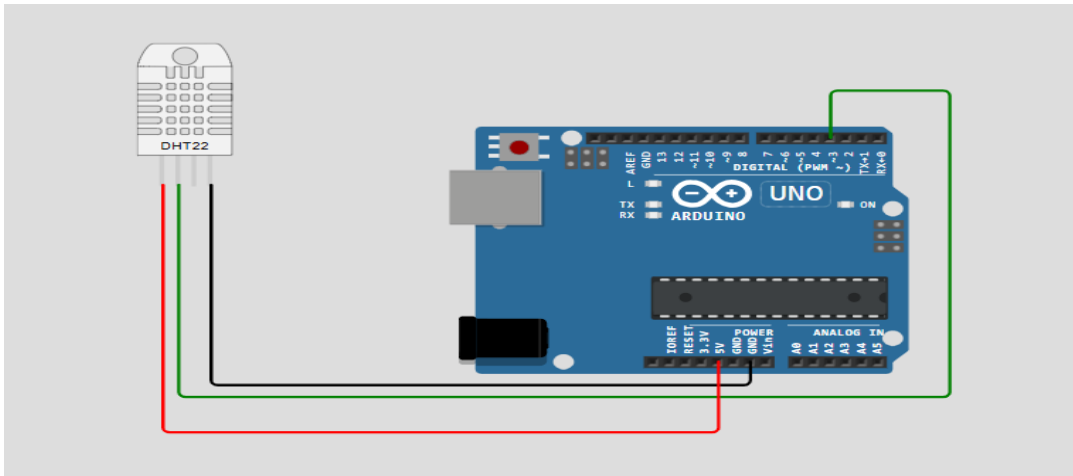
```

5. I compiled and uploaded the program to the Arduino Uno in the simulation.
6. I opened the **Serial Monitor** and observed the temperature and humidity values displayed continuously.

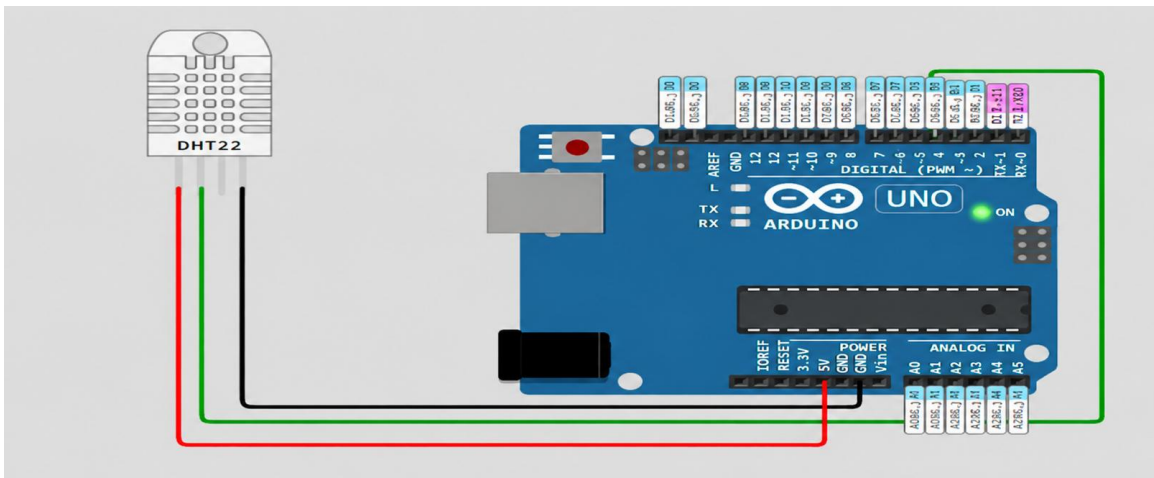
Result:

I successfully interfaced the DHT22 sensor with the Arduino Uno and observed real-time **temperature and humidity readings** displayed on the serial monitor.

Before:



After:



```
=====
Experiment 4: DHT22 Sensor Interface
Name   : KIRAN KUMAR PIKKI
Roll No : 25M11MC234
=====
Temperature = 24.00 °C, Humidity = 40.00 %
Temperature = 24.00 °C, Humidity = 40.00 %
```

Conclusion:

From this experiment, I understood how to interface digital sensors like the DHT22 with Arduino. I learned to use sensor libraries, read environmental parameters, and display them

through the serial monitor, which is useful in IoT-based weather monitoring and automation systems.

Experiment 5

Aim:

To interface an OLED with Arduino and write a program to print temperature and humidity readings on it.

Requirements:

- ☐ DHT22 sensor
- ☐ Arduino Uno microcontroller board (ATmega328p)
- ☐ Jumper wires
- ☐ SSD1306 OLED display

Description:

In this experiment, I connected an OLED display and DHT22 sensor to the Arduino Uno. The DHT22 measured the current temperature and humidity, while the OLED showed the readings. This experiment helped me understand I²C communication and how sensor data can be displayed, which is useful for IoT applications such as weather monitoring and home automation.

Procedure:

1. I opened www.wokwi.com in my browser and selected the **Arduino Uno** microcontroller.
2. In the simulation section, I added the required components (DHT22 sensor, SSD1306 OLED, jumper wires, and Arduino Uno) by clicking the “+” symbol.
3. I made the following connections using jumper wires:
 - a) I connected **VCC pin of DHT22** to **5V power supply** of Arduino.
 - b) I connected **SDA pin of DHT22** to **digital pin 3** of Arduino.
 - c) I connected **GND pin of DHT22** to **Ground (GND)** of Arduino.
 - d) I connected the **OLED display** as follows:
 - I. OLED DATA → Analog pin **A4**
 - II. OLED CLK → Analog pin **A5**
 - III. OLED VIN → **5V power**
 - IV. OLED GND → **Ground**
4. I downloaded and included the **Adafruit SSD1306**, **Adafruit GFX**, and **DHT sensor libraries** in the Arduino IDE.

5. I wrote the following program in **sketch.ino**:

```
/*
  Experiment 5: Interfacing OLED with Arduino Uno
  Purpose: To print temperature and humidity readings on OLED
  Name: KIRAN KUMAR PIKKI
  Roll No: 25M11MC234
*/

#include <Adafruit_Sensor.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>
#include <DHT_U.h>
#include <Wire.h>

// OLED settings
Adafruit_SSD1306 display(128, 64, &Wire);

// DHT22 settings DHT
Sensor(3, DHT22);

void setup() {
  Serial.begin(9600);
  Sensor.begin();

  // Initialize OLED
  if(!display.begin(SSD1306_SWITCHCAPVCC,0x3C)){
    Serial.println("SSD1306 allocation failed");
    for(;;);
  }
  display.clearDisplay();
  display.setTextColor(WHITE);

  // Show student details first
  display.setTextSize(1);
  display.setCursor(0, 10);
  display.println("Experiment 5");
  display.println("OLED with DHT22");
  display.println("-----");
  display.println("Name: KIRAN KUMAR PIKKI");
  display.println("Roll No: 25M11MC234");
  display.display();

  delay(3000); // Show info for 3 seconds
```

```

        display.clearDisplay();
    }
    void loop() {
        float tempr = Sensor.readTemperature(); float
        humid = Sensor.readHumidity();

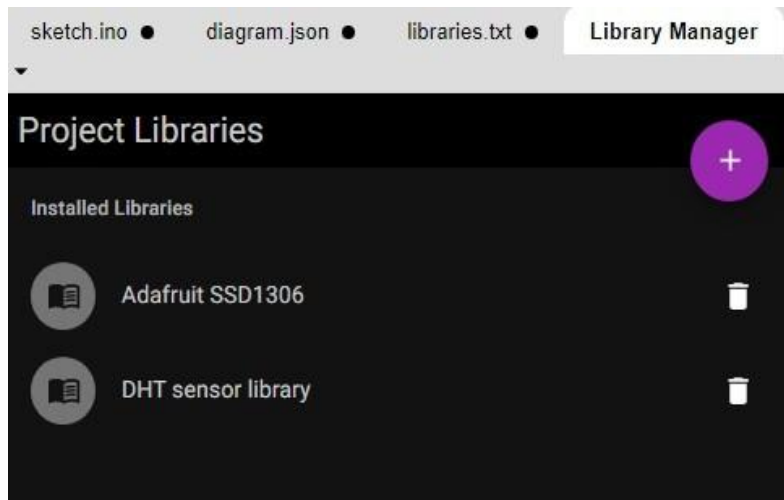
        // Display sensor readings
        display.setTextSize(1);
        display.setCursor(0, 10);
        display.print("Temp: ");
        display.print(tempr);
        display.println(" C");

        display.print("Humidity: ");
        display.print(humid);
        display.println(" %");

        display.display();
        delay(1000);    // Update every second
        display.clearDisplay();
    }

```

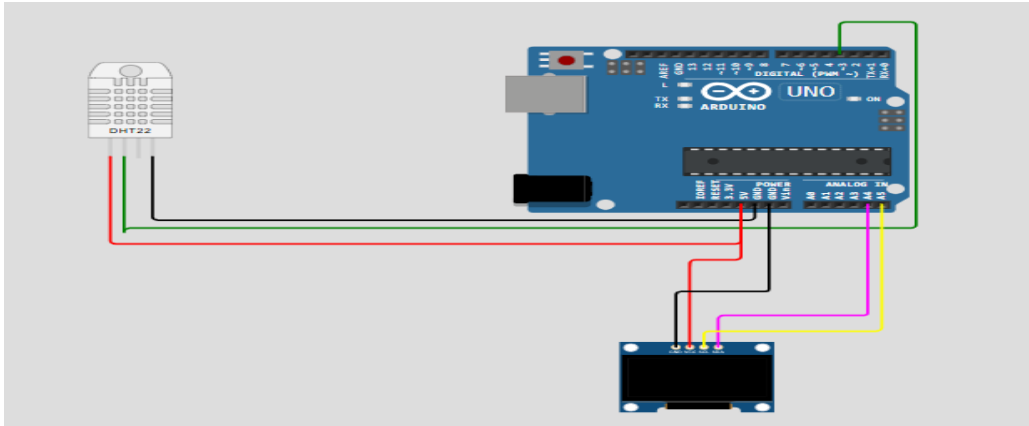
6. I compiled and uploaded the program to the Arduino Uno in the simulation.
7. I observed that the OLED displayed the real-time **temperature and humidity values** from the DHT22 sensor.



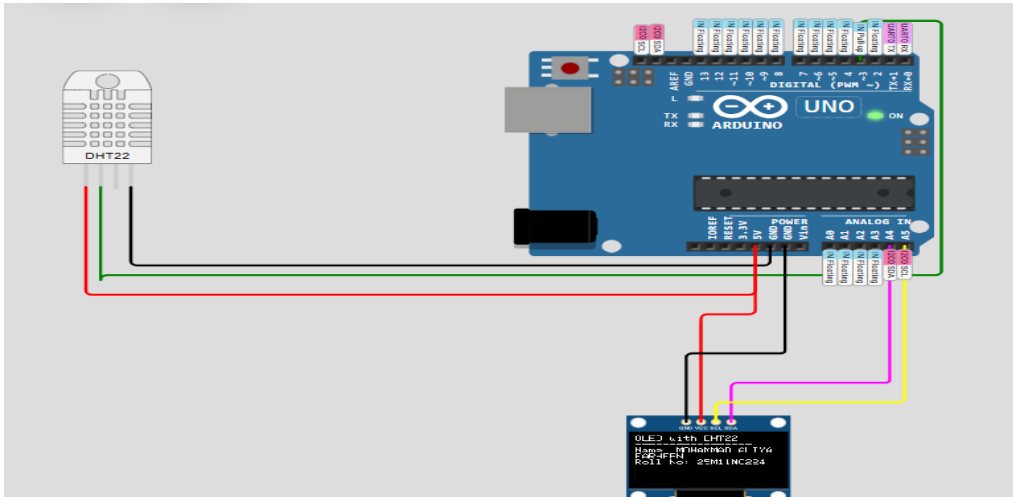
Result:

I successfully interfaced the OLED display with the Arduino Uno and observed that the real-time temperature and humidity readings from the DHT22 sensor were displayed on the OLED screen.

Before:



After:



Conclusion:

Through this experiment, I learned how to interface and program an OLED display with Arduino using the I²C protocol. I also understood how to collect real-time data from a sensor and display it on a graphical interface, which is an essential part of IoT applications.